

AS - PROYECTO EXÁMEN – BASE DE DATOS II

1. Alcance general del proyecto

Desarrollar un sistema Cliente/Servidor completo utilizando uno de los siguientes **stacks tecnológicos**:

- Node.js + Express + ORM
- ASP.NET Core + Entity Framework
- Python + ORM
- Go + framework HTTP
- u otro stack equivalente (framework + ORM/ODM)

La base de datos puede ser: **PostgreSQL, MySQL, MariaDB, MongoDB.**

2. Requisitos de Base de Datos

2.1 Modelo de datos

- Definir **mínimo 8 tablas o colecciones** relacionadas en el ORM/ODM del stack.
- Incluir obligatoriamente una tabla/colección **usuarios** para autenticación.
- Identificar al menos una **tabla principal** del sistema (la que se usará en la operación transaccional y/o reportes principales).

2.2 Auditoría y soft delete

- Todas las tablas o colecciones deben incluir campos:
 - createdAt
 - updatedAt
- La tabla principal debe incluir además:
 - deletedAt para implementar **soft delete** (el registro no se elimina físicamente, se marca con fecha de eliminación y las consultas normales del sistema deben considerar este campo para no mostrar registros eliminados como activos).

2.3 Lógica avanzada (función / trigger / ORM)

- Implementar una **lógica de negocio automática** que se ejecute en respuesta a operaciones sobre los datos (por ejemplo: actualización de totales, registro de auditoría adicional, bitácora de cambios, control de stock, etc.), usando:
 - **SQL** (función almacenada o trigger), o
 - el **framework/ORM** (hook, signal, middleware u otro mecanismo equivalente).

2.4 Transacciones

- Implementar al menos **una operación compleja** que:
 - Involucre más de una tabla o colección (por ejemplo: cabecera + detalle).
 - Use una **transacción** con inicio, commit y rollback en caso de error, usando la API de transacciones del ORM o del motor de base de datos.
-

3. Backend (API + Framework + ORM)

3.1 Modelos / Entidades

- Definir modelos/entidades para todas las tablas o colecciones en el framework/ORM.
- Incluir createdAt y updatedAt en todos los modelos.
- Incluir deletedAt en el modelo de la tabla principal y ajustar las consultas de negocio para respetar el soft delete.

3.2 APIs CRUD

Para **cada tabla o colección** se deben implementar al menos los siguientes endpoints:

- Lista de registros (GET).
- Obtener registro por ID (GET).
- Crear registro (POST).
- Actualizar registro (PUT).
- Eliminar registro (DELETE).

Condiciones:

- En la **tabla principal**, el endpoint de eliminar debe realizar **soft delete** (marcar deletedAt en lugar de borrar físicamente).

- Todas las APIs CRUD deben estar **protegidas con JWT**, excepto **una única API de listado público** (GET de una tabla seleccionada, por ejemplo productos) que se usará en la vista pública del frontend.
-

4. Seguridad (Login, bcrypt, JWT)

- La tabla/colección **usuarios** debe almacenar la contraseña en forma encriptada usando **bcrypt**.
 - El backend debe implementar:
 - Un **mecanismo de registro de usuarios** que guarde contraseñas encriptadas.
 - Un **mecanismo de autenticación/login** que valide las credenciales, genere un **JWT** y lo devuelva al cliente.
 - Debe existir un **middleware o guard** que:
 - Reciba el JWT enviado por el cliente.
 - Verifique la validez del token.
 - Permita o rechace el acceso a las rutas protegidas según el resultado.
-

5. Frontend

El frontend debe:

- Implementar una **vista de login** que consuma el mecanismo de autenticación del backend, reciba el JWT y lo almacene (por ejemplo en `localStorage` o mediante cookie, según el stack).
 - Enviar el JWT en todas las peticiones a endpoints protegidos del backend (por header o cookie, según el diseño).
 - Implementar **una vista pública** que consuma la única API pública de listado definida en el backend (por ejemplo, lista de productos) y funcione sin login.
 - Implementar **vistas CRUD protegidas** (listar, crear, editar, eliminar) para al menos **3 tablas diferentes**, consumiendo las APIs protegidas con JWT.
 - Mostrar en la interfaz el efecto del **soft delete** en la tabla principal (no mostrar registros eliminados como activos o marcarlos claramente como inactivos).
 - Incluir funcionalidad de **cerrar sesión** que elimine el JWT y bloquee el acceso a las vistas protegidas.
-

6. Reportes

6.1 Reporte HTML con parámetros y `window.print()`

- Vista de reporte en el frontend (HTML) que consuma datos desde el backend.
- El reporte debe recibir parámetros de fecha desde el frontend:
 - `fecha_inicio`
 - `fecha_fin`
- El backend debe filtrar los datos usando estos parámetros (con SQL o con el ORM).
- La vista debe permitir generar un PDF usando `window.print()`.

6.2 Reporte PDF generado en backend

- Endpoint en el backend que genere un **PDF** con datos de la base de datos.
 - El frontend debe permitir descargar ese PDF mediante un botón o enlace.
-

7. Entregables

Cada estudiante debe entregar:

- Código fuente del **backend** y del **frontend**, sin incluir carpetas de librerías o dependencias (por ejemplo: sin `node_modules`, sin entornos virtuales de Python, sin carpetas de dependencias de `.NET` o equivalentes en otros frameworks).
 - Un **video de demostración corto** donde se muestre:
 - Login y uso de JWT.
 - Vista pública consumiendo la API pública.
 - Algunas vistas CRUD protegidas.
 - Uso del soft delete en la tabla principal.
 - Generación del reporte HTML con `window.print()`.
 - Descarga del reporte PDF generado en el backend.
-

8. Rúbrica de Calificación (Total: 100 puntos)

Criterio / Ítem	Puntaje máximo	Puntaje obtenido
Tablas, colecciones y modelos	20	
Definición de al menos 8 tablas o colecciones en el ORM/ODM	6	
Relaciones correctas entre tablas/colecciones (PK, FK, referencias o equivalentes)	6	
Tabla/colección <code>usuarios</code> definida correctamente para login (campos necesarios)	4	
Identificación y uso de una tabla principal en la lógica del sistema (transacción y/o reportes)	4	
Auditoría y soft delete	15	
Campo <code>createdAt</code> presente y usado en todas las tablas/colecciones	5	
Campo <code>updatedAt</code> presente y actualizado en todas las tablas/colecciones	5	
Campo <code>deletedAt</code> en la tabla principal y soft delete funcionando correctamente	5	
Lógica avanzada y transacciones	15	
Lógica de negocio automática con función SQL, trigger o mecanismo avanzado del ORM (hook, signal, middleware, etc.)	7	
Operación compleja con transacción, commit y rollback funcionando y coherente con la lógica del sistema	8	
Backend – APIs y protección de rutas	20	
CRUD completo (lista, detalle, crear, actualizar, eliminar) implementado para todas las tablas/colecciones	10	
Todas las APIs privadas protegidas correctamente con JWT mediante middleware	5	
Implementación correcta de una única API pública de listado para la vista pública del frontend	5	
Seguridad de autenticación	10	
Contraseñas de usuarios almacenadas usando bcrypt	4	
Generación y validación de JWT en el flujo de autenticación y uso correcto del middleware que lo verifica	6	
Frontend – funcionalidad	10	
Vista pública funcionando, consumiendo la API pública sin requerir login	3	
Vista de login, almacenamiento del JWT y cierre de sesión correctos	4	
Vistas CRUD protegidas para al menos 3 tablas distintas, consumiendo APIs protegidas con JWT	3	
Reportes	5	

Criterio / Ítem	Puntaje máximo	Puntaje obtenido
Reporte HTML con parámetros de fecha y generación de PDF mediante <code>window.print()</code>	3	
Reporte PDF generado en backend y descargable desde el frontend	2	
Diseño y presentación del cliente web	5	
Organización visual básica, formularios utilizables, tablas legibles y navegación clara	5	
TOTAL	100	